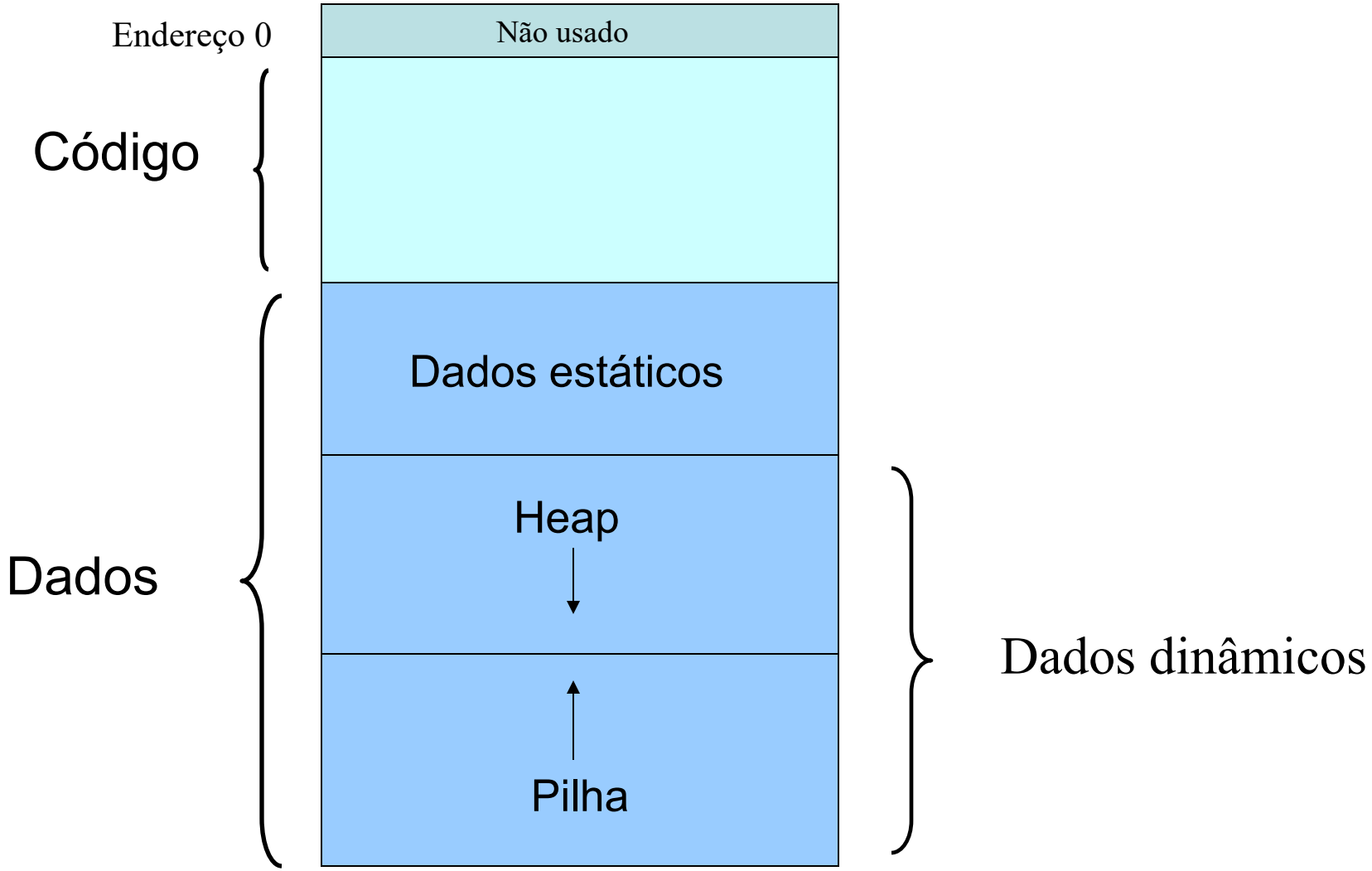
UDESC

Compiladores

Ambiente em Tempo de Execução
(*Runtime environment*)

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Organização da Memória



UDESC

Dados Estáticos

A área de dados estática é reservada no início da execução do programa e liberada apenas no fim de sua execução (*e.g.* constantes, variáveis globais e variáveis locais declaradas com o modificador *static* em linguagem C).

UDESC

Dados Dinâmicos

Pilha – Área de armazenamento temporário onde é armazenado o registro de ativação das funções.

Heap – Área destinada à alocação dinâmica de memória, permitindo ao programador reservar e liberar blocos de memória quando necessário. (*e.g.* áreas de memória reservadas pelas funções *malloc* e liberadas pela função *free* em linguagem C).

UDESC

Registro de Ativação

As informações necessárias para execução de uma função/procedimento são gerenciadas utilizando um bloco de memória chamado registro de ativação, podem fazer parte do registro de ativação: parâmetros, endereço de retorno e variáveis locais.

UDESC

Variáveis estáticas e variáveis locais

```
int foo (int n)
{
    static int x = 0;
    x++;
    return x+n;
}
int main()
{
    int x;
    x = foo(3);
    return 0;
}
```

```
"foo":
    push  rbp
    mov   rbp, rsp
    mov   DWORD PTR [rbp-4], edi
    mov   eax, DWORD PTR "x.0"[rip]
    add   eax, 1
    mov   DWORD PTR "x.0"[rip], eax
    mov   edx, DWORD PTR "x.0"[rip]
    mov   eax, DWORD PTR [rbp-4]
    add   eax, edx
    pop   rbp
    ret
```

```
"main":
    push   rbp
    mov    rbp, rsp
    sub    rsp, 16
    mov    edi, 3
    call   "foo"
    mov    DWORD PTR [rbp-4], eax
    mov    eax, 0
    leave
    ret
```

Chamadas de Funções / Procedimentos
ABI (*Application Binary Interface*)

```
int fat (int n)
{
      if (n <= 1)
              return 1;
      return n * fat(n-1);
}
int main()
{
    int x;
    x = fat(3);
    return 0;
}
```

```
"fat":
    push  ebp
    mov   ebp, esp
    cmp   DWORD PTR [ebp+8], 1
     jg     .L2
     mov  eax, 1
     jmp   .L3
.L2:
    mov   eax, DWORD PTR [ebp+8]
    sub    eax, 1
    push  eax
    call     "fat"
    add    esp, 4
    imul    eax, DWORD PTR [ebp+8]
.L3:
     leave
     ret
```

⟹

```
mov esp, ebp
pop  ebp
```

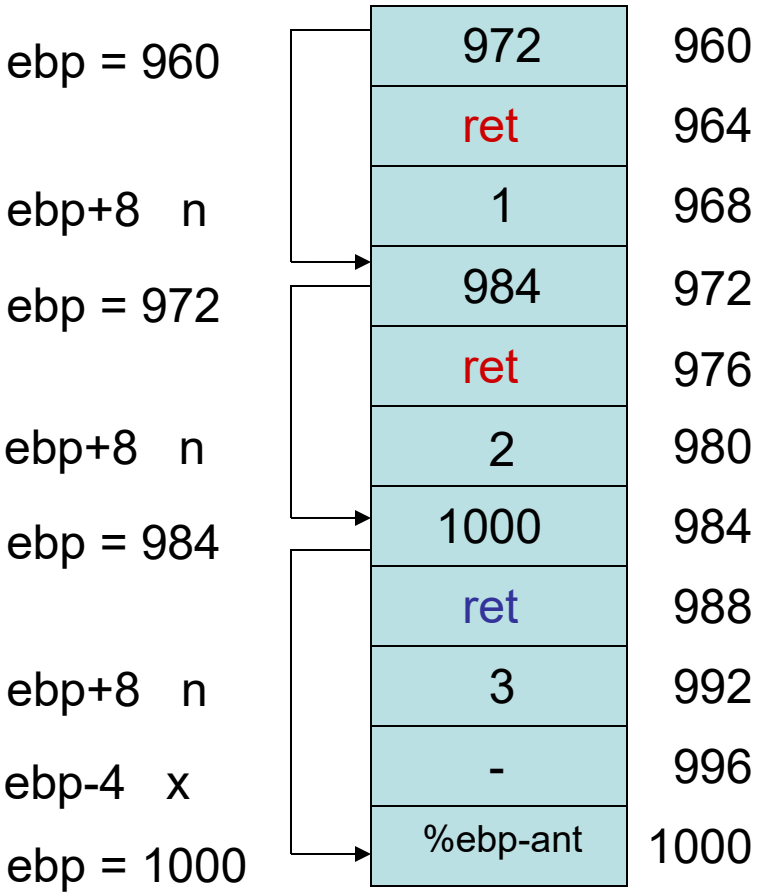
```
"main":
    push  ebp
    mov   ebp, esp
    sub    esp, 4
    push  3
    call    "fat"
    add    esp, 4
    mov   DWORD PTR [ebp-4], eax
    mov   eax, 0
    leave
    ret
```

gcc -m32 -mpreferred-stack-boundary=2 (alinhamento de 4 bytes, $\log_2 4$)

Chamadas de Funções / Procedimentos

ABI (*Application Binary Interface*)

```
int fat (int n)
{
    if (n <= 1)
        return 1;
    return n * fat(n-1);
}
int main()
{
    int x;
    x = fat(3);
    return 0;
}
```



Chamadas de Funções / Procedimentos
ABI (*Application Binary Interface*)

```
int fat (int n)
{
      if (n <= 1)
            return 1;
      return n * fat(n-1);
}
int main()
{
      int x;
      x = fat(3);
}
```

```
"fat":
   push   rbp
   mov    rbp, rsp
   sub    rsp, 16 *
   mov    DWORD PTR [rbp-4], edi
   cmp    DWORD PTR [rbp-4], 1
   jg     .L2
   mov    eax, 1
   jmp    .L3
.L2:
   mov    eax, DWORD PTR [rbp-4]
   sub    eax, 1
   mov    edi, eax
   call   "fat"
   imul   eax, DWORD PTR [rbp-4]
.L3:
   leave
   ret
```



```
mov rsp, rbp
pop rbp
```

```
"main":
    push   rbp
    mov    rbp, rsp
    sub    rsp, 16 *
    mov    edi, 3
    call   "fat"
    mov    DWORD PTR [rbp-4], eax
    mov    eax, 0
    leave
    ret
```

* A ABI é definida com um alinhamento de 16 bytes

UDESC

Garbage Collector

(Coletor de Lixo)

Coletores de lixo são uma forma automática de gerenciamento de memória, responsável por identificar objetos inacessíveis e liberar automaticamente a memória por eles ocupada no *heap*. Foi proposto por John McCarthy em 1959 para simplificar o gerenciamento de memória da linguagem LISP. Evita problemas como:

- Ponteiros pendentes (dangling pointers);
- Erros de dupla liberação;
- Vazamento de memória (*memory leaks*).

UDESC

Reference Counting
(Contadores de Referência)

Reserva alguns bits junto ao dado para indicar quantas referências existem ao objeto. Quando o contador chega a zero, o objeto pode ser imediatamente desalocado.

- Simples, mas inclui um overhead em cada operação de atribuição.
- Problemas para lidar com referências circulares.

UDESC

Mark and Sweep
(marcação e varredura)

- Suspende a execução do programa e utiliza os apontadores para determinar quais os objetos estão em uso.
- Objetos que não são referenciados são considerados lixo e sua área de memória é liberada.